

PROG39051 Final Project Report

By Marco Hidalgo and Miyuki Randle

Introduction

As the game industry expands and technology advances, the competition and need for video games to be successful has greatly expanded. With the large number of games that are released each year, it is becoming increasingly difficult for games to stand out and often will fall under the radar upon release. This can be very a devastating issue for developers and publishers trying to create games for players to enjoy.

This project would be particularly useful to indie game developers because they create games on their own time, with their own money, and it usually a game they are very passionate about so it would be a terrible loss for their game to financially and/or critically flop. Whereas larger game corporations are more likely able to handle such losses and have the funds to move on to either improve the game or start a completely new project.

The project aims to employ machine learning techniques that can predict how well a game will succeed before release, so that developers can potentially make the necessary improvements that would statistically correlate to game popularity. We aim to examine aspects of games (i.e. associated genres, associated categories, available platforms, price) that might lead to greater success (i.e. more positive reviews, more purchases, more playtime) upon release. Popularity of certain types of games can also vary based on what is popular at that time, which can fluctuate.

This model would help developers and publishers to not waste time and money just to release something that ultimately fails, since they would be able to include aspects into their game which are more popular.

Literature Review

For implementation, we will compare an interpretable baseline model such as linear regression with a stronger nonlinear model such as Random Forest, since the baseline will show which features contribute most directly to success while the tree-based model can capture complex interactions among game attributes.

Linear Regression

According to our class notes, “Linear regression is a supervised [machine] learning technique to find the linear relationship between two variables. It is a model that attempts to describe the linear relationship between the independent and dependent variables. The equation of a linear regression line is $y = mx + b$, where x is the independent variable and y is the dependent variable. The slope of the line is m , and b is the intercept” (Mahmoud, 2026, 3). It is considered the “the simplest and most classic linear method for regression” (Müller & Guido, 2017, p. 53).

Our secondary research does not show as much information on this technique as it does on Random Forest, most likely due to the former’s sheer simplicity.

Former master’s student Gregorius Henry Wirawan and associate professor Dr. Gede Putra Kusuma from Bina Nusantara University state that a Multiple Linear Regression method was used by Haocheng Zhang of San Jose State University to predict sales of games, as well as to investigate various factors related with game sales, and that it received a residual-squared score of 0.6756 (Wirawan & Kusuma, 2024, p. 348, 351). “The results indicated that linear models might not be suitable for predicting the number of players” (Wirawan & Kusuma, 2024, p. 351).

Federal University of Minas Gerais professors Henrique Pinto, Jussara M. Almeida, and Marcos A. Gonçalves observe a linear regression model proposed in 2010 by University of Szeged faculty head Gábor Szabó and Research Institutes of Sweden visiting scientist Dr. Bernardo Huberman and find out their proposal “states that the future popularity [of online content] is related to the early one by a constant factor α , which depends only on the target and reference dates and not on any specific information of the content itself” (Pinto et al., 2013, p. 365). Pinto et al. also state the following: “While the reported results produced by the Szabo and Huberman (S-H) model are reasonably accurate, especially given its simplicity, the model does have shortcomings. In particular, two different pieces of content may have very similar popularity at the same reference date and yet exhibit distinct popularity behaviors thereafter” (Pinto et al., 2013, p. 365).

Random Forest

Researcher KV Vidath Vardhan, in his paper for the International Journal for Multidisciplinary Research, describes Random Forest as “a smart and diverse group of decision trees working together to make accurate predictions” (Vidath Varhan, 2024, p. 3). The model essentially builds a collection of trees “with different subsets of the data and features” (Vidath Varhan, 2024, p. 3), then the trees produce a “vote” on the outcome, and the most popular “vote” becomes the final prediction. The approach this model uses also helps overcome overfitting in data that contains excessive noise (outliers) and, as a result, “improves accuracy and robustness” (Vidath Varhan, 2024, p. 3). During his research, Random Forest also received an accuracy score of 89.38%, higher than both Logistic Regression and the regular Decision Tree classifier (Vidath Varhan, 2024, p. 5).

University of Surrey research fellow Dr. Abdulrahman Kerim and Hacettepe University associate professor Dr. Burakay Genç state that Random Forest’s training process also results in variance being reduced (Kerim & Genç, 2025, p. 545).

Former National College of Ireland master’s student Aashish Prasad states the following in his master’s thesis: “Random Forest is best in handling imbalanced [datasets] and have shown high accuracy because of its ensemble working principle” (Prasad, 2019, p. 8).

Research scientist Michal Trněný states in his master’s thesis for Masaryk University that a new Random Forest model can also be trained for attributes that contain missing values (Trněný, 2017, p. 41). However, he also states that “Random Forest has higher tendency to over-estimate games with an actually low number of average concurrent players. We consider this a bigger issue as it may give a false hope to a developer who ends up creating a game not enough players will buy, possibly resulting in not having its development even funded” (Trněný, 2017, p. 41). Nevertheless, during his research, much like in Mr. Vidath Vardhan’s, Random Forest also scored the highest, “with 0.7 correlation and 72 % [Root Relative Squared Error], closely followed by [Support Vector Machine] with 0.7 correlation and 74 % [Root Relative Squared Error]. Gaussian Process and even a simple linear model give only slightly worse results, while [Recursive Partitioning and Regression Trees] scores significantly below all other algorithms” (Trněný, 2017, p. 44).

Implementation

Data Description

For implementation we are using the Steam Store Games dataset obtained from Kaggle. This dataset provides information and statistics about various video game assets for all released video games on the Steam platform as of May 2019. There are nearly 30,000 video game entry rows and 18 feature columns of this dataset, they include:

- appid: int64 (Ex. 10)
- name: object (Ex. 'Counter-Strike')
- release_date: object (Ex. '2000-11-01')
- english: int64 (Language support: 1 if is in English)
- developer: object (Ex. 'Arkane Studios')
- publisher: object (Ex. 'Ubisoft')
- platforms: object (Ex. 'windows;mac;linux')
- required_age: int64 (Ex. 0 if Unrated or unsupplied)
- Categories: object (Ex. 'Single-player;Multi-player;Steam Achievements')
- genres: object (Ex. 'Indie;Strategy')
- steamspy_tags: object (Ex. 'Action;FPS;Multiplayer')
- achievements: object (Number of in-games achievements, if any, Ex. 15)
- positive_ratings: int64 (Ex. 1563)
- negative_ratings: int64 (Ex. 288)
- average_playtime: int64 (Ex. 22494)
- median_playtime: int64 (Ex. 6502)
- owner : object (Estimated number of owners. Contains lower and upper bound, Ex. '2000000-5000000')
- price: float64 (Ex. 5.79)

Linear Regression

The first ML technique that was highlighted in the literature to be effective for predicting video game success was Linear Regression. While not necessarily the ideal best technique for our data and goals, it was emphasized that Linear Regression is an excellent starting point as a baseline model. It can be used in an ensemble or as a baseline comparison against more complex non-linear ML models, such as Random Forest, which provides much greater focus on model-based importance.

The data was cleaned, ensuring to check for and handle any missing values. The summary of null values showed that just the 'developer' and 'publisher' had some missing values. Since only a small number of rows are missing categorical values, I chose to fill them with 'Unknown' to preserve the dataset size while maintaining data consistency.

It is also important to ensure that the data is unbiased to create an effective model. For Linear Regression, class imbalance is not a concern since it does not involve classification problems, however it can arise in a different balancing issue causing skewed distribution. This issue became especially apparent with our type of data and particular dataset. Skewed distribution stems the Linear Regression becoming dominated by the few major games with high stats but perform poorly on more average games. To counteract this problem, I applied log transformations to transform data columns that could be affected.

With the goal of our project and the information organization from the Steam dataset in mind, our ML pipeline needed to have some extensive feature engineering to ensure implementation made sense and allow us to use more of the available features in a meaningful way.

There were multiple features in the data that could equated to demonstrate the success of a game, such as the number of owners, number of positive ratings or average playtime. To effectively target overall success, I created a combined success scoring metric to account for all success factors. The owners and playtime are scaled, while a ratio is created for the positive vs negative ratings, then all 3 metrics are added together with weights assigned to each one. The weights are assigned to prioritize popularity (0.5), while still accounting for player satisfaction (0.3) and engagement (0.2), reflecting a balanced definition of game success.

The data also contained many categorical features that we wanted to be able to consider which might correlate to success, such as genre and category. In order to use these types of features they needed to be encoded and have dummy/indicator values put in place. This makes each genre/category listed as its own separate feature, with a value of 0 or 1 to

indicate if the game entry is part of that genre or category. This allows us to use the categorical features in our model in a meaningful way.

The X and y were then defined, with y the target as the Success Score and X as all the potential features we will consider correlations to. At this stage, we also made sure to drop the now unnecessary columns before moving on, which includes the columns used in the success score, features that don't logically make sense and any non-numeric features no longer needed as well.

Feature selection was performed using a combination of variance thresholding and correlation analysis. First, low-variance features were removed using a variance threshold of 0.001 to eliminate features with little variability, like among one-hot encoded categorical variables.

Next, a correlation filter was applied to retain only features with an absolute correlation greater than 0.01 with the target variable (success_score). Features used in constructing the target variable, such as owners, playtime, and rating ratio, were excluded to prevent data leakage, as mentioned above.

This process resulted in a reduced and more informative feature set, improving model efficiency and interpretability.

In order to optimize the performance of the model, polynomial features were also added which allows linear models to map non-linear relationships present in the data. This ensures that all important relationships linear or not can be captured for the model.

Since the data has a large variety of value scales per column it was important that next, we made sure the data was standardized using StandardScalar to make sure features can contribute equally.

After splitting the data into train and test data the model could finally be created. To best optimize the model, the regularized version of Linear Regression, known as Ridge Regression, was used instead. Using Ridge helped prevent overfitting and overall produced better R2 results. The Ridge Alpha parameter was also optimized using a GridSearchCV to tune the Alpha to ensure the most effective value is chosen.

Random Forest

The second ML technique that was highlighted in the literature to be effective for predicting video game success was Random Forest. As stated before, it is a more complex non-linear ML model than Linear Regression and provides much greater focus on model-based importance.

Much like with Linear Regression, the data was cleaned, ensuring to check for and handle any missing values. The same method to replace the null values that was used in Linear Regression was used here.

It is also important to ensure that the data is unbiased to create an effective model. Since we used RandomForestRegressor and not RandomForestClassification, class imbalance is not a concern since our model does not involve classification problems. It can, however, arise in a different balancing issue causing skewed distribution. This issue became especially apparent with our type of data and particular dataset. Certain features were dominated by a few major games with extremely high values. Such skewed distributions can cause the model to overfit to these outliers and reduce its accuracy on more average cases. To mitigate this problem, we applied log transformations to relevant columns to smooth large discrepancies and help the RandomForestRegressor learn more balanced patterns across the dataset.

After applying most of the same implementation criteria that was applied to Linear Regression (i.e. defining X and y, feature selection, normalization, train and test split), we modified the following settings to the RandomForestRegressor:

The maximum depth of the trees was set to 15, as the “None” default value for max_depth results in unlimited depth, which results in the trees continuously growing until all leaves either are pure or contain less than min_sample_split samples.

The minimum number of samples required to be at a leaf node was set to 10, as the default value of 1 for min_samples_leaf results in the same issues as the “None” default value for maximum depth.

The number of features to consider when looking for the best split was set to 0.5, as we would like to discard features that are irrelevant to our goal (i.e. english).

Evaluation

Linear Regression

A Baseline was created for comparison to the actual model to confirm model effectiveness. The Baseline produced a score of approximately 0.016, which is significantly lower than the model R^2 score of 0.72. This indicates a strong and effective learning model has been created.

Cross-validation produced a lower R^2 score than the final test set, suggesting that performance may vary depending on the data split. This indicates some sensitivity to the dataset and highlights the importance of using cross-validation for a more robust estimate of model performance.

Overall, the model demonstrates strong and consistent performance across both training and test datasets. The small gap between training and test metrics (MAE, RMSE, and R^2) indicates that the model generalizes well and does not suffer from significant overfitting. With a test R^2 of 0.72, the model is able to explain a substantial portion of the variance in the target variable, suggesting it is an effective baseline for predicting video game success.

The residual plot analysis shows that the errors are centered around zero, indicating no major systematic bias in predictions. The residual distribution is approximately normal, suggesting that the normality assumption is reasonably met. However, the residuals vs predicted plot exhibits a funnel-shaped pattern, indicating heteroscedasticity and potential model misspecification. This suggests that the linear regression model may not fully capture underlying relationships, and improvements could be made by incorporating non-linear features or alternative models.

Random Forest

The same Baseline as Linear Regression was used to compare to the actual RandomForestRegressor model to confirm its effectiveness. The score of approximately 0.016 is also significantly lower than the RandomForestRegressor model's R^2 score of 0.73, indicating a strong and effective ML model.

In this case, cross-validation produced a higher R^2 score than the final test set, once again suggesting that performance may vary depending on the data split. This indicates some sensitivity to the dataset and highlights the importance of using cross-validation for a more robust estimate of model performance.

Overall, just like with Linear Regression, the RandomForestRegressor model demonstrates strong and consistent performance across both training and test datasets. The small gap between training and test metrics (MAE, RMSE, and R^2) indicates that the model generalizes well and does not suffer from significant overfitting. With a test R^2 of 0.73, the model is able to explain a substantial portion of the variance in the target variable, suggesting it is an effective baseline for predicting video game success.

Just like with Linear Regression, the residual plot analysis shows that the errors are centered around zero, indicating no major systematic bias in predictions. The residual distribution is approximately normal, suggesting that the normality assumption is reasonably met. However, the residuals vs predicted plot exhibits a funnel-shaped pattern, indicating heteroscedasticity and potential model misspecification. This suggests that the linear regression model may not fully capture underlying relationships, and improvements could be made by incorporating non-linear features or alternative models.

Conclusion

In conclusion, the results of this project show that RandomForestRegressor performs slightly better than Linear Regression for predicting the success of games. This suggests that RandomForestRegressor may be a more suitable algorithm for similar tasks.

References

- Emina Salkanović, Nejla Zukorlić, Lamija Oković, & Kečo, D. (2024). A Data Analysis of Steam's Game Catalog and Diverse Recommendation Strategies. *International Journal of Computer Applications*, 186(54), 39–49. <https://doi.org/10.5120/ijca2024924261>
- Hu, W., Wang, Y., & Xia, R. (2024). Machine learning-based steam platform game's popularity analysis. *Proceedings of the 1st International Conference on Data Science and Engineering*, 582–588. <https://doi.org/10.5220/0012853800004547>
- Kerim, A., & Genç, B. (2025). *Mobile games success and failure: mining the hidden factors*. *Neural Computing & Applications*, 37(2), 543–557. <https://doi.org/10.1007/s00521-022-07154-z>
- Mahmoud, E. (2026). 4. *How to build a machine learning Model in Python* [PDF slides]. Slate. <https://slate.sheridancollege.ca/d2l/le/content/1462846/viewContent/18908836/View>
- Müller, A. C., & Guido, S. (2017). *Introduction to Machine Learning with Python: A Guide for Data Scientists*. O'Reilly.
- Pfau, J., Debus, M., Juul, J., Lundedal Hammar, E., Canossa, A., & Seif El-Nasr, M. (2022). Predicting Success Factors of Video Game Titles and Companies. In J. Baalsrud Hauge, E. van der Spek, R. McCall, & B. Göbl (Eds.), *Entertainment Computing - ICEC 2022* (Vol. 13477, pp. 269–282). Springer International Publishing AG. https://doi.org/10.1007/978-3-031-20212-4_22
- Pinto, H., Almeida, J. M., & Gonçalves, M. A. (2013). Using early view patterns to predict the popularity of YouTube videos. *Proceedings of the Sixth ACM International Conference on Web Search and Data Mining*, 365–374. <https://doi.org/10.1145/2433396.2433443>
- Prasad, A. (2019). *Estimating Video Game Success using Machine Learning*. Unpublished. <https://doi.org/10.13140/RG.2.2.14389.01767>
- Trněný, M. (2017). *Machine learning for predicting success of video games* [Master's thesis, Masaryk University]. Masaryk University Information System. https://web.archive.org/web/20240723050310/https://is.muni.cz/th/k2c5b/diploma_thesis_trneny.pdf
- Vidath Varhan, K. (2024). Machine learning models in predicting gaming popularity: A comparative analysis. *International Journal For Multidisciplinary Research*, 6(3). <https://doi.org/10.36948/ijfmr.2024.v06i03.14362>

Wirawan, G. H., & Kusuma, G. P. (2024). Predicting the number of video game players on the steam platform using machine learning and time lagged features. *International Journal of Advanced Computer Science and Applications*, 15(12).

<https://doi.org/10.14569/ijacsa.2024.0151237>

Yu, H. (2024). A Comprehensive Analysis of Game Success: Synergy Between Quality, Brand, and Market. *Applied and Computational Engineering*, 110(1), 22–28.

<https://doi.org/10.54254/2755-2721/110/2024melb0101>

Ziyang, J. (2021). *Predicting the Popularity of Independent Video Games on the Steam Platform*. <https://doi.org/10.17615/aaj0-x494>